

+ TEMA 05

CALIDAD DE DATOS

- Datos nulos
- Outliers
- Formatos incorrectos
- Normalización





+ INTRODUCCIÓN **DE LA SESIÓN**

- + La calidad de los datos impacta directamente en el desempeño de los modelos de IA.
- + Datos incorrectos o incompletos pueden generar resultados erróneos o sesgados.
- + Antes de analizar o modelar, es indispensable evaluar y mejorar la calidad del dataset.
- + En esta sesión, abordaremos los principales problemas de calidad de datos y cómo tratarlos.



+
DATOS
NULOS

¿QUÉ SON LOS DATOS NULOS?

- Son valores faltantes o no registrados dentro de un dataset.
- Se representan comúnmente como NaN, None, NULL o campos vacíos.
- **Causas habituales:** errores en la recolección, fallas de hardware o información incompleta.
- **Impacto crítico:** hasta el 80 % del tiempo de un proyecto de análisis puede destinarse a la limpieza de estos valores.
- **Riesgo principal:** muchos modelos y algoritmos no aceptan datos con valores nulos, lo que puede invalidar el análisis o el entrenamiento.

Name	Age	Salary	Country
Anna	28	45000	NaN
John	NULL	32000	USA
Sophia	35	—	—
Michael	NULL	NULL	Germany

ESTRATEGIAS DE MANEJO: ELIMINACIÓN VS. IMPUTACIÓN

ELIMINACIÓN (DROPPING)

- **Rápida pero riesgosa:** implica perder filas o columnas, lo que puede eliminar información valiosa y sesgar el análisis.
- Se recomienda solo cuando menos del 5 % de los datos son nulos y el patrón de ausencia es aleatorio.

	A	B	C	D
1	3	1	2	3.5
2	4	NAN	2	3.0
3	2	NAN	3	3.5
4	2	NAN	3	3.5

IMPUTACIÓN (FILLING)

- **Estrategia preferida:** reemplaza los valores nulos por estimaciones informadas (media, mediana, moda o modelos). Esto permite conservar el dataset y mejorar el desempeño predictivo del modelo.

	A	B	C	D
1	3	1	2	3.5
2	4	— 3 →	2	3.0
3	2	— 3 →	1	3.1
4	2	2.1	2.1	3.5



MEJOR PRÁCTICA

Primero analiza el contexto, luego elige la estrategia y finalmente documenta la decisión.



MÉTODOS DE IMPUTACIÓN (TÉCNICAS)

1

MEDIA / MEDIANA

Sustituyen por un valor central. Son rápidas y simples, pero reducen la variabilidad del conjunto de datos.

2

MODA

Reemplaza con el valor más frecuente. Adecuada para variables categóricas.

3

FORWARD / BACKWARD FILL

Usado en series temporales: propaga el último valor válido hacia adelante o atrás.

4

KNN (K-NEAREST NEIGHBORS)

Imputa a partir de los valores de observaciones similares. Más sofisticado, pero computacionalmente costoso.

5

INTERPOLACIÓN

Estima valores entre puntos conocidos, común en datos ordenados o temporales.

6

MODELOS PREDICTIVOS

Entrenan un modelo específico para predecir los valores faltantes, logrando mayor precisión a costa de mayor complejidad.

CÓDIGO PANDAS: IMPUTACIÓN

Práctica

```
# Detectar nulos  
print(df.isnull().sum()) # Cuántos por columna  
print(df.isnull().any()) # Hay alguno?
```

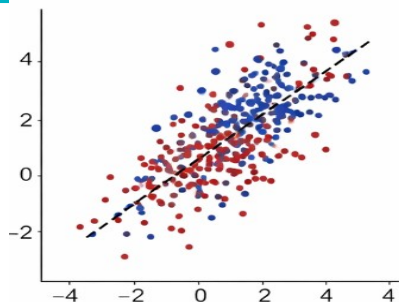
```
# Imputar forward fill (series)  
df['temperatura'].fillna(method='ffill', inplace=True)
```

```
# Imputar con media  
df['edad'].fillna(df['edad'].mean(), inplace=True)  
# Imputar con mediana (mejor para outliers)  
df['precio'] = df['precio'].fillna(df['precio'].median())
```



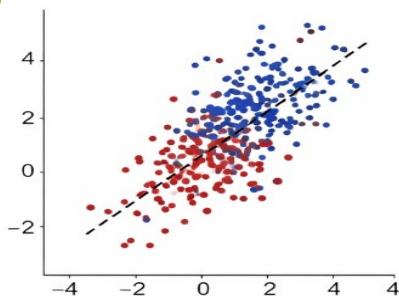
PATRONES DE DATOS NULOS

MCAR (Missing Completely at Random)



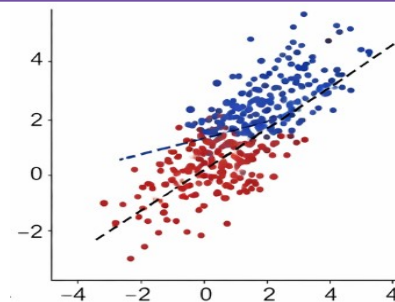
- Los datos faltan sin un patrón identificable ni relación con otras variables.
- La imputación es relativamente segura y suele introducir poco sesgo.

MAR (Missing at Random)



- La ausencia de datos está relacionada con otras variables observadas.
- Requiere análisis previo para elegir una estrategia de imputación adecuada.

MNAR (Missing Not at Random)

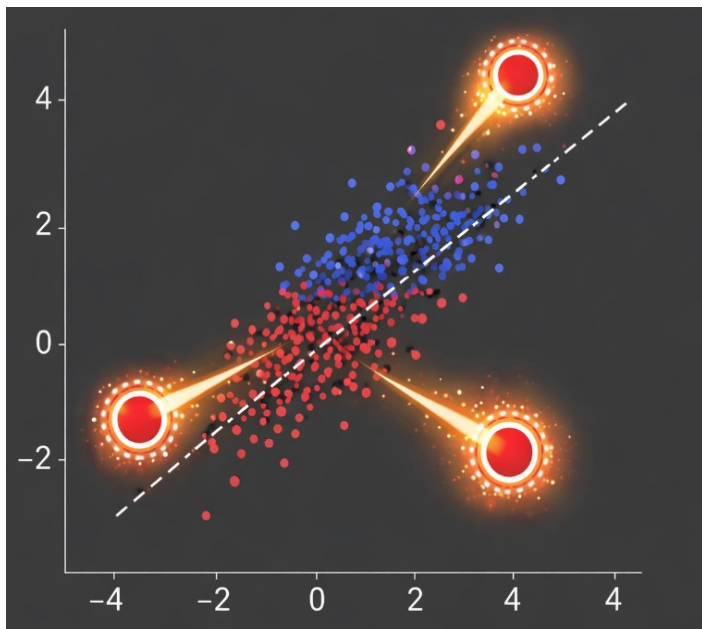


- La omisión no es aleatoria y depende del valor faltante o de decisiones intencionales.
- Es indispensable incorporar contexto y supuestos antes de imputar.

Regla clave: siempre investiga por qué faltan los datos antes de aplicar cualquier método de imputación.

+
OUTLIERS

¿QUÉ SON OUTLIERS?



- 01  Valores que se desvían significativamente del patrón normal.
- 02  Ejemplos fáciles de identificar: edad = 250 años; salario = -US\$1,000,000.
- 03  La pregunta clave en los outliers es la siguiente: ¿error o información valiosa?
- 04  Impacto: pueden arruinar modelos que afectan la media y/o las correlaciones.
- 05  Estos se pueden aplicar para la detección de fraude, anomalías médicas y ciberataques.

TIPOS DE OUTLIERS



GLOBAL OUTLIERS

- Observaciones con desviación extrema respecto a todo el dataset.



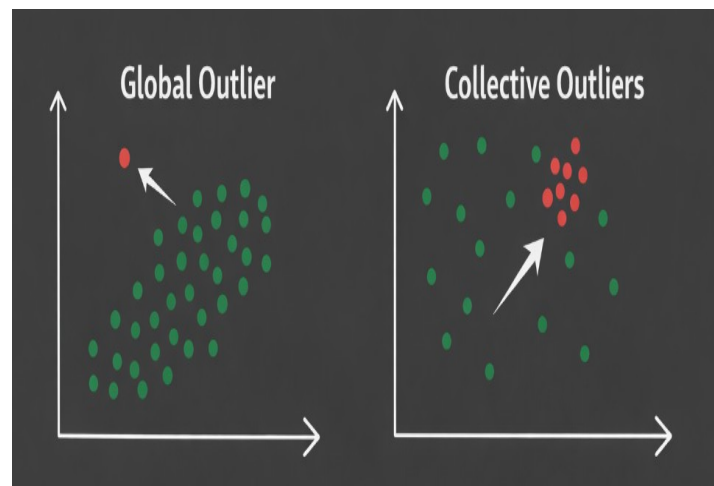
LOCAL OUTLIERS

- Parecen normales a nivel global, pero son anómalos dentro de su vecindad (ejemplo: tienda con ventas bajas en un barrio de alto poder adquisitivo).



COLLECTIVE OUTLIERS

- Conjuntos de observaciones que, tomadas en grupo, forman una anomalía (ejemplo: un spike inusual en una serie temporal).



MÉTODOS DE DETECCIÓN DE OUTLIERS

REGLA 3 - SIGMA	IQR (RANGO INTERCUARTÍLICO)	ISOLATION FOREST	LOF (LOCAL OUTLIER FACTOR)	Z-SCORE
Si un valor está fuera de las 3 desviaciones estándar, es outlier.	Se establece el límite superior sumando al tercer cuartil (Q3) 1.5 veces el valor del rango intercuartílico (IQR).	Es un algoritmo caracterizado por ser rápido y efectivo, ideal para manejar datos de alta dimensión	Este método se enfoca en detectar valores atípicos "locales" analizando la densidad de los datos en su vecindario.	Método para detectar outliers que estandariza los datos mediante la fórmula $(x-\mu)/\sigma$. Cualquier valor cuyo resultado absoluto ($ Z $) sea mayor a 3 se considera atípico.

ISOLATION FOREST: ALGORITMO EFICIENTE

- Aísla observaciones anómalas mediante la construcción de árboles aleatorios.
- Los outliers requieren menos particiones (splits) para ser aislados, por lo que presentan caminos más cortos en los árboles.

Ventajas

- + Complejidad casi lineal $O(n)$: alto rendimiento computacional.
- + No requiere cálculo de distancias pairwise.
- + Escalable y eficaz en espacios de alta dimensionalidad.
- + Capaz de detectar outliers globales y locales.

Aplicación práctica

```
from sklearn.ensemble import IsolationForest  
iso_forest = IsolationForest(contamination=0.05)  
outlier_labels = iso_forest.fit_predict(data)
```

¿QUÉ HACER CON OUTLIERS?: DECISIONES



MÉTODO 1: ELIMINAR

Debe aplicarse con cautela, ya que puede implicar pérdida de información crítica, especialmente cuando los outliers representan fraude, fallas técnicas o eventos médicos relevantes.



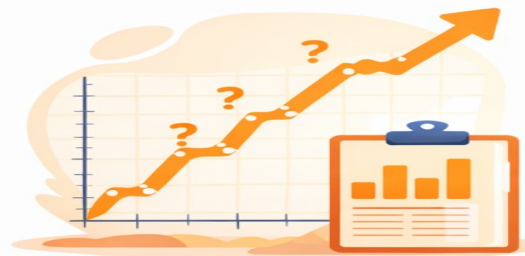
MÉTODO 2: TRANSFORMAR/CREAR

Consiste en suavizar o ajustar los valores extremos. En series temporales, puede incluir la propagación del último valor válido o transformaciones como log o winsorization.



MÉTODO 3: INVESTIGAR

Enfoque más riguroso: analizar la causa del valor atípico y, si corresponde, estimar su valor usando métodos como vecinos más cercanos (KNN) o modelos específicos.



+
FORMATOS
INCORRECTOS

TIPOS DE ERRORES DE FORMATO

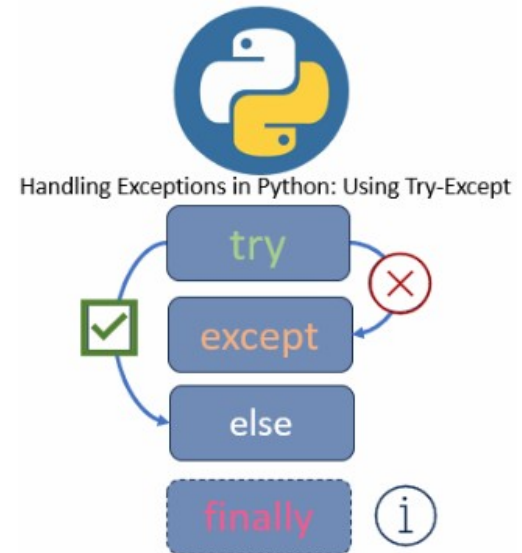
Tipos de datos incorrectos	Inconsistencias	Rangos inválidos	Caracteres extraños	Valores duplicados o contradictorios
Variables almacenadas en un formato inadecuado (ejemplo: edad como texto en lugar de número).	Uso de distintos formatos para una misma variable (ejemplo: fechas en DD/MM/YYYY y DD-MM-YYYY).	Valores fuera de límites lógicos o definidos (ejemplo: edad negativa o porcentajes mayores al 100 %).	Presencia de espacios, acentos o símbolos no esperados que afectan el procesamiento.	Registros con el mismo identificador (ID), pero información diferente.

CONVERSIÓN DE TIPOS (TYPE CASTING)

Problema común: DataFrame carga edad como “string” y eso hace imposible hacer cálculos.

```
>>>
>>> # Detectar tipo
... print(df.dtypes) # Ver tipos por columna
...
... # Convertir
... df['edad'] = df['edad'].astype(int) # string → entero
... df['precio'] = df['precio'].astype(float) # string → decimal
... df['fecha'] = pd.to_datetime(df['fecha']) # string → datetime
```

- Usa try-except para manejar errores en conversión.



LIMPIEZA Y NORMALIZACIÓN DE TEXTO

Elimina espacios al inicio/final - `str.strip()`

Espacios

Remueve acentos si no son necesarios.

Acentos

Detecta "Juan" y "juan" como mismo valor.

Duplicados



Mayúscula

Estandariza a minúsculas - `str.lower()`

Caracteres especiales

Elimina símbolos no alfanuméricos.

Aplicación práctica:

```
>>> # Limpiar texto
... df['ciudad'] = df['ciudad'].str.strip().str.lower()
... # Resultado: " Barcelona " → "barcelona"
... 
```

VALIDACIÓN DE RANGOS Y RESTRICCIONES

Definir restricciones:

- Edad: 0-120.
- Email: contiene @.
- Teléfono: 10 dígitos.

```
>>> # Detectar violaciones
... invalid_ages = df[(df['edad'] < 0) | (df['edad'] > 120)]
... print(f"Edades inválidas: {len(invalid_ages)}")
...
... # Validar email
... valid_emails = df['email'].str.contains('@', regex=False)
... invalid_emails = df[~valid_emails]
```

➤ Reemplaza o elimina valores inválidos según contexto.



ESTANDARIZACIÓN: FORMATOS CONSISTENTES

Problema: datos Inconsistentes.

- Fechas: "01/01/2024" junto con "2024-01-01" y "1 ene 2024".

```
# Estandarizar fechas
df['fecha'] = pd.to_datetime(df['fecha'], format='mixed')
df['fecha_formateada'] = df['fecha'].dt.strftime('%Y-%m-%d')

# Estandarizar teléfono: +56-9-1234-5678
df['telefono'] = df['telefono'].str.replace('[^0-9]', '', regex=True)
```

- Un solo formato garantiza análisis sin errores.



+
NORMALIZACIÓN

¿POR QUÉ NORMALIZAR?

Las variables con rangos muy distintos pueden sesgar o confundir a los algoritmos.

Ejemplo: edad (0-100) vs. salario (0-10 millones).

Algunos algoritmos requieren variables en la misma escala para funcionar correctamente.



Mejora la convergencia de los modelos y puede acelerar el entrenamiento significativamente.

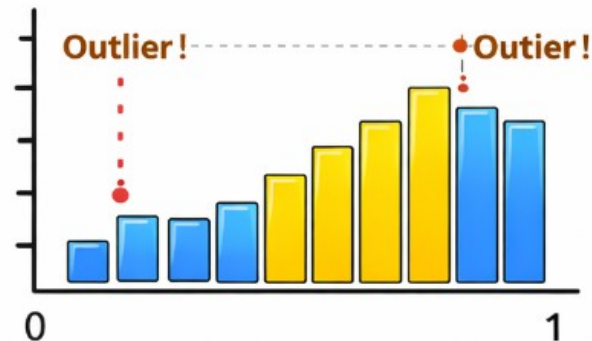
Es esencial para algoritmos basados en distancia o gradiente como KNN, redes neuronales y SVM.

TÉCNICAS PRINCIPALES

Min-Max Normalization [0,1]



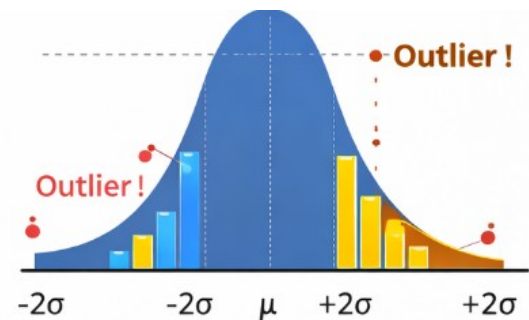
- Fórmula: $(x - \min) / (\max - \min)$
- Mantiene un rango conocido [0,1].
- Sensible a outliers.



Estandarización (Z-Score)



- Fórmula: $(x - \mu) / \sigma$, donde μ es media y σ es desviación estándar.
- Media = 0, desviación estándar = 1.
- Más robusta frente a outliers.
- Rango infinito.



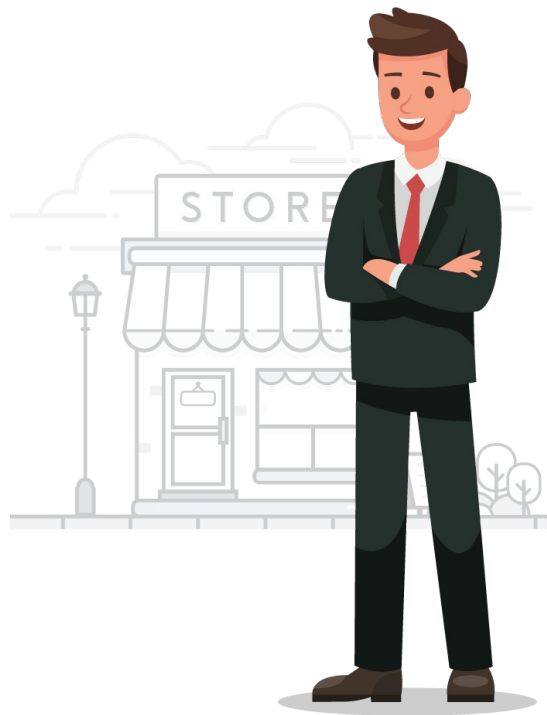
ESTANDARIZACIÓN: FORMATOS CONSISTENTES

```
Python 3.14 (64-bit) X + v

Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> from sklearn.preprocessing import MinMaxScaler, StandardScaler
...
... # Min-Max Normalization [0, 1]
... scaler_minmax = MinMaxScaler()
... df_normalized = pd.DataFrame(scaler_minmax.fit_transform(df))
...
... # Standardization (Z-Score)
... scaler_std = StandardScaler()
... df_standardized = pd.DataFrame(scaler_std.fit_transform(df))
... |
```

```
Python 3.14 (64-bit) X + v

Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> # Correcto: Fit solo en train
... scaler.fit(X_train)
... X_train_scaled = scaler.transform(X_train)
... X_test_scaled = scaler.transform(X_test) # NO fit!
```



GUÍA: 4 TÉCNICAS ESENCIALES

1

Detecta valores faltantes e imputa usando métodos como media, KNN o modelos predictivos.

2

Identifica outliers con Isolation Forest y decide si tratarlos, transformarlos o investigar su causa.

3

Asegura la calidad de los datos: valida tipos, limpia texto y estandariza formatos.

4

Normaliza variables numéricas mediante Min-Max o Z-Score, según el algoritmo y el objetivo.

Datos limpios → modelos más precisos y confiables.

+ **CONCLUSIONES**



x



x



+ CONCLUSIONES

- + La calidad de datos es un factor crítico para el éxito de soluciones IA.
- + Identificar y tratar datos nulos evita pérdida de información relevante.
- + El análisis de outliers permite tomar decisiones informadas sobre el dataset.
- + Corregir formatos incorrectos garantiza consistencia y confiabilidad.
- + La normalización prepara los datos para un análisis y modelado más efectivo.





x



x



+

BIBLIOGRAFÍA **MÁS REFERENCIAS**

- + Abburi, C. K. (2024). *Optimizing big data quality management for national-scale projects: Strategies and frameworks*. International Journal of Intelligent Systems and Applications in Engineering, 12(3), 7187–7196.
<https://ijisae.org/index.php/IJISAE/article/view/7187>
- + Boukerche, A., Zheng, L., & Alfandi, O. (2020). *Outlier detection: Methods, models, and classification*. ACM Computing Surveys, 53(3), Article 55. <https://doi.org/10.1145/3381028>
- + Emmanuel, T., Maupong, T., Mpoeleng, D., Semong, T., Mphago, B., & Tabona, O. (2021). *A survey on missing data in machine learning*. Journal of Big Data, 8, Article 140. <https://doi.org/10.1186/s40537-021-00516-9>
- + Intriago, M. G., & Cevallos, R. A. (2025). *Impacto de la normalización de datos en la precisión de modelos de aprendizaje supervisado*. RIEMAT: Revista de Investigaciones en Energía, Medio Ambiente y Tecnología, 10(2), 59–79.
<https://revistas.utm.edu.ec/index.php/Riemat/article/view/7853>



- + Kelkar, B. A. (2022). *Missing data imputation: A survey*. International Journal of Decision Support System Technology, 14(1), 1–20. <https://doi.org/10.4018/IJDS.ST.292446>
- + Miao, X., Wu, Y., Chen, L., Gao, Y., & Yin, J. (2022). *An experimental survey of missing data imputation algorithms*. IEEE Transactions on Knowledge and Data Engineering, 35(7), 6630–6650. <https://doi.org/10.1109/TKDE.2022.3186498>
- + Rodríguez González, J., & Ugalde Saborio, E. (2021). *Impacto de la estandarización y escalado: Factor para predicción de costos en proyectos a través de una red neuronal artificial*. Ingeniare. Revista Chilena de Ingeniería, 29(2), 265–275. <https://doi.org/10.4067/S0718-33052021000200265>



ISIL+